



Algoritmos y Estructuras de Datos II

Práctica 1: Divide y Vencerás

Profesor:

José Ramón Hoyos Barceló

Alumnos:

Vinicius Paredes Cabral

Pablo Herrero Tornero

Grupo: 3

Subgrupos: 3.1 (Pablo) y 3.4 (Vinicius)

Email:

pablo.h.t@um.es
vinicius.p.c@um.es

Fecha: 19 de marzo de 2025

Índice

1. Introducción	2
2. Diseño de la solución divide y vencerás	2
2.1. Descripción del problema	2
2.2. Estructuras de datos	2
2.3. Funciones básicas del esquema algorítmico	2
2.4. Pseudocódigo	3
2.5. Explicación del algoritmo	4
3. Análisis teórico del tiempo de ejecución	5
3.1. Análisis de la función contarAparicionesConsecutivas	5
3.2. Análisis de la función principal	5
3.3. Caso mejor	5
3.4. Caso peor	6
3.5. Orden de ejecución	6
4. Implementación	6
4.1. Estructuras de datos	6
4.2. Función auxiliar	7
4.3. Función principal	7
4.4. Función main	8
4.5. Correspondencia con el esquema algorítmico	9
5. Validación del algoritmo	9
5.1. Solución directa	9
5.2. Casos de prueba	10
5.3. Resultados de la validación	11
5.4. Conclusiones de la validación	11
6. Estudio experimental del tiempo de ejecución	11
6.1. Configuración del experimento	11
6.2. Resultados experimentales	12
6.3. Análisis de los resultados	12
6.3.1. Comportamiento general	12
6.3.2. Diferencias entre tiempo real y tiempo de CPU	12
6.3.3. Análisis por tipo de caso	13
6.3.4. Escalabilidad	13
7. Contraste del estudio teórico y experimental	14
7.1. Comparación de complejidades	14
7.2. Discrepancias observadas	14
7.3. Justificación de las discrepancias	14
7.4. Conclusiones del contraste	15
8. Conclusiones	16
8.1. Conclusiones personales	16

1 Introducción

2 Diseño de la solución divide y vencerás

2.1 Descripción del problema

El problema consiste en encontrar una subcadena de longitud m que contenga el máximo número de apariciones consecutivas de un carácter C en una cadena A de longitud n . La solución debe devolver la posición inicial de la subcadena y el número de apariciones consecutivas del carácter C en ella.

2.2 Estructuras de datos

Para la implementación del algoritmo se utilizan las siguientes estructuras de datos:

- **String:** Para almacenar la cadena de entrada A .
- **Struct Resultado:** Una estructura que contiene:
 - **posicion:** Entero que indica la posición inicial de la subcadena óptima.
 - **apariciones:** Entero que indica el número de apariciones consecutivas del carácter C .

2.3 Funciones básicas del esquema algorítmico

El algoritmo divide y vencerás implementa las siguientes funciones básicas:

- **Divide:** La cadena se divide en dos mitades aproximadamente iguales.
- **Vence:** Se resuelve recursivamente el problema para cada mitad.
- **Combina:** Se combinan las soluciones de las subcadenas, considerando también las posibles soluciones que cruzan el punto medio.

2.4 Pseudocódigo

Algorithm 1 Encontrar Subcadena Óptima

```
1: function ENCONTRARSUBCADENAOPTIMA(cadena, inicio, fin, C, m)
2:   if fin - inicio + 1 < m then
3:     return {-1, 0}
4:   end if
5:   if fin - inicio + 1 = m then
6:     return contarAparicionesConsecutivas(cadena, inicio, fin, C, m)
7:   end if
8:   medio  $\leftarrow$  inicio + (fin - inicio) / 2
9:   izquierda  $\leftarrow$  encontrarSubcadenaOptima(cadena, inicio, medio, C, m)
10:  derecha  $\leftarrow$  encontrarSubcadenaOptima(cadena, medio + 1, fin, C, m)
11:  inicioLimite  $\leftarrow$  max(inicio, medio - m + 1)
12:  finLimite  $\leftarrow$  min(fin, medio + m - 1)
13:  limite  $\leftarrow$  contarAparicionesConsecutivas(cadena, inicioLimite, finLimite, C, m)
14:  if izquierda.apariciones  $\geq$  derecha.apariciones Y izquierda.apariciones  $\geq$  limite.apariciones then
15:    return izquierda
16:  else if derecha.apariciones  $\geq$  izquierda.apariciones Y derecha.apariciones  $\geq$  limite.apariciones then
17:    return derecha
18:  else
19:    return limite
20:  end if
21: end function
```

Algorithm 2 Contar Apariciones Consecutivas

```

1: function CONTARAPARICIONESCONSECUTIVAS(cadena, inicio, fin, C, m)
2:   res  $\leftarrow$  {inicio, 0}
3:   maxConsecutivas  $\leftarrow$  0
4:   posicionMax  $\leftarrow$  inicio
5:   for i  $\leftarrow$  inicio hasta fin - m + 1 do
6:     consecutivas  $\leftarrow$  0
7:     maxActual  $\leftarrow$  0
8:     for j  $\leftarrow$  i hasta i + m - 1 do
9:       if cadena[j] = C then
10:        consecutivas  $\leftarrow$  consecutivas + 1
11:        maxActual  $\leftarrow$  max(maxActual, consecutivas)
12:       else
13:        consecutivas  $\leftarrow$  0
14:       end if
15:     end for
16:     if maxActual > maxConsecutivas then
17:       maxConsecutivas  $\leftarrow$  maxActual
18:       posicionMax  $\leftarrow$  i
19:     end if
20:   end for
21:   res.posicion  $\leftarrow$  posicionMax
22:   res.apariciones  $\leftarrow$  maxConsecutivas
23:   return res
24: end function

```

2.5 Explicación del algoritmo

El algoritmo implementa una solución divide y vencerás con las siguientes características:

- **Divide:** La cadena se divide en dos mitades aproximadamente iguales en cada llamada recursiva.
- **Vence:**
 - Si la longitud de la subcadena es menor que m, no hay solución posible.
 - Si la longitud es igual a m, se cuenta directamente las apariciones consecutivas.
 - En caso contrario, se resuelve recursivamente para cada mitad.
- **Combina:**
 - Se obtienen las soluciones de las subcadenas izquierda y derecha.
 - Se busca una posible solución que cruce el punto medio (importante para no perder soluciones óptimas).
 - Se devuelve la solución con el máximo número de apariciones consecutivas.

La función auxiliar `contarAparicionesConsecutivas` se encarga de:

- Buscar en una subcadena dada todas las posibles subcadenas de longitud m .
- Para cada subcadena, contar el máximo número de apariciones consecutivas del carácter C .
- Mantener registro de la posición que da el máximo número de apariciones consecutivas.

3 Análisis teórico del tiempo de ejecución

3.1 Análisis de la función contarAparicionesConsecutivas

La función `contarAparicionesConsecutivas` tiene una complejidad de $O(n \cdot m)$, donde:

- n es la longitud de la subcadena a analizar
- m es la longitud de la subcadena objetivo

Esto se debe a que:

- El bucle exterior se ejecuta $n - m + 1$ veces
- Para cada iteración, el bucle interior se ejecuta m veces
- Las operaciones dentro de los bucles son de tiempo constante

3.2 Análisis de la función principal

Para analizar el tiempo de ejecución de la función principal, consideramos la ecuación de recurrencia:

$$T(n) = 2T(n/2) + O(n \cdot m)$$

Donde:

- $2T(n/2)$ representa las llamadas recursivas a cada mitad
- $O(n \cdot m)$ es el tiempo de la función auxiliar

3.3 Caso mejor

El caso mejor ocurre cuando:

- La subcadena óptima está completamente contenida en una de las mitades
- No es necesario explorar la región del límite

En este caso, la ecuación de recurrencia se simplifica a:

$$T(n) = 2T(n/2) + O(1)$$

Aplicando el teorema maestro:

$$a = 2, b = 2, f(n) = O(1)$$

$$\log_b(a) = \log_2(2) = 1$$

$$f(n) = O(1) = O(n^{\log_b(a)})$$

Por lo tanto, el caso mejor tiene una complejidad de $O(n)$.

3.4 Caso peor

El caso peor ocurre cuando:

- La subcadena óptima cruza el punto medio
- Es necesario explorar todas las regiones

En este caso, la ecuación de recurrencia es:

$$T(n) = 2T(n/2) + O(n \cdot m)$$

Aplicando el teorema maestro:

$$a = 2, b = 2, f(n) = O(n \cdot m)$$

$$\log_b(a) = \log_2(2) = 1$$

$$f(n) = O(n \cdot m) = O(n^{\log_b(a)})$$

Por lo tanto, el caso peor tiene una complejidad de $O(n \cdot m \cdot \log n)$.

3.5 Orden de ejecución

En resumen:

- **Caso mejor:** $O(n)$
- **Caso peor:** $O(n \cdot m \cdot \log n)$
- **Caso promedio:** $O(n \cdot m \cdot \log n)$

La complejidad espacial es $O(\log n)$ debido a la profundidad de la recursión.

4 Implementación

4.1 Estructuras de datos

Listing 1: Estructura Resultado

```

1 // Estructura para almacenar el resultado
2 struct Resultado {
3     int posicion;           // Posición inicial de la subcadena
4     int apariciones;        // Número de apariciones consecutivas
5 };

```

4.2 Función auxiliar

Listing 2: Función contarAparicionesConsecutivas

```
1 // Funci n auxiliar para contar apariciones consecutivas en una
  subcadena
2 Resultado contarAparicionesConsecutivas(const string& cadena, int
  inicio, int fin, char C, int m) {
3     Resultado res = {inicio, 0};
4     int maxConsecutivas = 0;
5     int posicionMax = inicio;
6
7     // Para cada posible posici n inicial de una subcadena de
      longitud m
8     for (int i = inicio; i <= fin - m + 1; i++) {
9         int consecutivas = 0;
10        int maxActual = 0;
11
12        // Contar apariciones consecutivas en la subcadena actual
13        for (int j = i; j < i + m; j++) {
14            if (cadena[j] == C) {
15                consecutivas++;
16                maxActual = max(maxActual, consecutivas);
17            } else {
18                consecutivas = 0;
19            }
20        }
21
22        // Actualizar m ximo si es necesario
23        if (maxActual > maxConsecutivas) {
24            maxConsecutivas = maxActual;
25            posicionMax = i;
26        }
27    }
28
29    res.posicion = posicionMax;
30    res.apariciones = maxConsecutivas;
31    return res;
32 }
```

4.3 Función principal

Listing 3: Función encontrarSubcadenaOptima

```
1 // Funci n principal de divide y vencer s
2 Resultado encontrarSubcadenaOptima(const string& cadena, int
  inicio, int fin, char C, int m) {
3     // Caso base: si la longitud es menor que m, no hay soluci n
4     if (fin - inicio + 1 < m) {
5         return {-1, 0};
6     }
```

```

7
8 // Si la longitud es igual a m, contar directamente
9 if (fin - inicio + 1 == m) {
10     return contarAparicionesConsecutivas(cadena, inicio, fin,
11         C, m);
12 }
13
14 // Dividir en dos mitades
15 int medio = inicio + (fin - inicio) / 2;
16
17 // Resolver recursivamente cada mitad
18 Resultado izquierda = encontrarSubcadenaOptima(cadena, inicio
19     , medio, C, m);
20 Resultado derecha = encontrarSubcadenaOptima(cadena, medio +
21     1, fin, C, m);
22
23 // Buscar en el l mite entre las dos mitades
24 int inicioLimite = max(inicio, medio - m + 1);
25 int finLimite = min(fin, medio + m - 1);
26 Resultado limite = contarAparicionesConsecutivas(cadena,
27     inicioLimite, finLimite, C, m);
28
29 // Combinar resultados
30 if (izquierda.apariciones >= derecha.apariciones && izquierda
31     .apariciones >= limite.apariciones) {
32     return izquierda;
33 } else if (derecha.apariciones >= izquierda.apariciones &&
34     derecha.apariciones >= limite.apariciones) {
35     return derecha;
36 } else {
37     return limite;
38 }
39 }

```

4.4 Función main

Listing 4: Función main

```

1 int main() {
2     string cadena;
3     int n, m;
4     char C;
5
6     // Leer entrada
7     cout << "Introduce la longitud de la cadena (n): ";
8     cin >> n;
9
10    cout << "Introduce la longitud de la subcadena (m): ";
11    cin >> m;
12
13    cout << "Introduce el car cter a buscar (C): ";

```

```
14     cin >> C;
15
16     cout << "Introduce la cadena: ";
17     cin >> cadena;
18
19     // Medir tiempo de ejecución
20     struct timeval ti, tf;
21     double tiempo;
22     gettimeofday(&ti, NULL);
23
24     // Resolver el problema
25     Resultado resultado = encontrarSubcadenaOptima(cadena, 0, n
        -1, C, m);
26
27     gettimeofday(&tf, NULL);
28     tiempo = (tf.tv_sec - ti.tv_sec)*1000 + (tf.tv_usec - ti.
        tv_usec)/1000.0;
29
30     // Mostrar resultado
31     cout << "\nResultado:" << endl;
32     cout << "Posición inicial: " << resultado.posicion + 1 <<
        endl;
33     cout << "Apariciones consecutivas: " << resultado.apariciones
        << endl;
34     cout << "Tiempo de ejecución: " << tiempo << " milisegundos"
        << endl;
35
36     return 0;
37 }
```

4.5 Correspondencia con el esquema algorítmico

La implementación sigue fielmente el esquema divide y vencerás:

- **Divide:** Se implementa en la línea donde se calcula el punto medio y se divide la cadena en dos partes.
- **Vence:** Se implementa en las llamadas recursivas a `encontrarSubcadenaOptima` para cada mitad.
- **Combina:** Se implementa en la parte final de la función, donde se comparan las soluciones de las subcadenas y se busca en el límite.

5 Validación del algoritmo

5.1 Solución directa

Para validar el algoritmo divide y vencerás, se ha implementado una solución directa que explora todas las posibles subcadenas de longitud m . Esta solución tiene una complejidad de $O(n \cdot m)$ y sirve como referencia para verificar la corrección de nuestro algoritmo.

Listing 5: Solución directa

```
1 Resultado solucionDirecta(const string& cadena, char C, int m) {
2     Resultado res = {0, 0};
3     int maxConsecutivas = 0;
4     int posicionMax = 0;
5
6     // Explorar todas las posibles subcadenas de longitud m
7     for (int i = 0; i <= cadena.length() - m; i++) {
8         int consecutivas = 0;
9         int maxActual = 0;
10
11        // Contar apariciones consecutivas en la subcadena actual
12        for (int j = i; j < i + m; j++) {
13            if (cadena[j] == C) {
14                consecutivas++;
15                maxActual = max(maxActual, consecutivas);
16            } else {
17                consecutivas = 0;
18            }
19        }
20
21        // Actualizar m ximo si es necesario
22        if (maxActual > maxConsecutivas) {
23            maxConsecutivas = maxActual;
24            posicionMax = i;
25        }
26    }
27
28    res.posicion = posicionMax;
29    res.apariciones = maxConsecutivas;
30    return res;
31 }
```

5.2 Casos de prueba

Se han realizado las siguientes pruebas para validar el algoritmo:

1. Caso del ejemplo del enunciado:

- Cadena: "cddabecdacc"
- Longitud (n): 10
- Subcadena (m): 5
- Carácter: 'c'
- Resultado esperado: posición 6, 2 apariciones consecutivas

2. Casos límite:

- Cadena vacía
- Cadena con longitud igual a m

- Cadena con longitud menor que m
- Cadena con todos los caracteres iguales
- Cadena sin apariciones del carácter buscado

3. Casos especiales:

- Subcadena óptima al inicio
- Subcadena óptima al final
- Subcadena óptima en el medio
- Subcadena óptima que cruza el punto medio

5.3 Resultados de la validación

La siguiente tabla muestra los resultados de la validación comparando la solución divide y vencerás con la solución directa:

Caso de prueba	n	m	DyV	Directa
Ejemplo enunciado	10	5	(6,2)	(6,2)
Cadena vacía	0	5	(-1,0)	(-1,0)
Todos iguales	10	5	(0,5)	(0,5)
Sin apariciones	10	5	(0,0)	(0,0)

5.4 Conclusiones de la validación

- El algoritmo divide y vencerás produce resultados idénticos a la solución directa en todos los casos de prueba.
- La implementación maneja correctamente los casos límite y especiales.
- La solución es robusta y mantiene la corrección incluso en casos extremos.

6 Estudio experimental del tiempo de ejecución

6.1 Configuración del experimento

Se han realizado mediciones del tiempo de ejecución para tres tipos de casos:

- **Caso favorable:** Cadena con todos los caracteres iguales al buscado
- **Caso desfavorable:** Cadena sin apariciones consecutivas del carácter buscado
- **Caso promedio:** Cadena con distribución aleatoria de caracteres

Para cada tipo de caso, se han probado diferentes tamaños de entrada desde 100 hasta 100.000.000 caracteres, siguiendo una progresión de potencias de 10.

6.2 Resultados experimentales

La siguiente tabla muestra los tiempos de ejecución (en segundos) para cada caso:

Tipo	Tamaño (n)	Real	User	Sys
favorable	100	0.005	0.001	0.002
favorable	1000	0.002	0.001	0.001
favorable	10000	0.004	0.002	0.001
favorable	100000	0.017	0.016	0.001
favorable	1000000	0.125	0.122	0.001
favorable	10000000	1.227	1.216	0.006
favorable	100000000	12.038	11.950	0.058
desfavorable	100	0.002	0.000	0.001
desfavorable	1000	0.002	0.000	0.001
desfavorable	10000	0.003	0.002	0.001
desfavorable	100000	0.014	0.012	0.001
desfavorable	1000000	0.121	0.119	0.001
desfavorable	10000000	1.206	1.197	0.005
desfavorable	100000000	12.455	12.242	0.080
promedio	100	0.002	0.000	0.001
promedio	1000	0.003	0.001	0.001
promedio	10000	0.004	0.002	0.001
promedio	100000	0.015	0.013	0.001
promedio	1000000	0.129	0.125	0.002
promedio	10000000	1.215	1.203	0.007
promedio	100000000	12.686	12.238	0.078

6.3 Análisis de los resultados

6.3.1. Comportamiento general

Los resultados muestran que:

- El tiempo de ejecución crece de manera aproximadamente lineal con el tamaño de la entrada para valores grandes.
- Para tamaños pequeños (hasta $n=10.000$), el crecimiento es más lento que lo esperado teóricamente.
- Para tamaños grandes ($n \geq 1.000.000$), el crecimiento se aproxima a un factor 10 cuando el tamaño se multiplica por 10.
- Los tres tipos de casos (favorable, desfavorable y promedio) muestran tiempos de ejecución similares, con ligeras variaciones.

6.3.2. Diferencias entre tiempo real y tiempo de CPU

El tiempo real puede ser significativamente mayor que el tiempo de CPU (user + sys) por varias razones:

- **Operaciones de E/S:** La lectura de archivos grandes requiere operaciones de entrada/salida que son manejadas por el sistema operativo.
- **Multitarea:** El sistema operativo puede estar ejecutando otros procesos en paralelo que compiten por los recursos.
- **Gestión de memoria:** La asignación y liberación de memoria para cadenas grandes puede requerir operaciones del sistema que no se reflejan en el tiempo de usuario.
- **Caché y memoria virtual:** Los accesos a memoria que no están en caché o requieren paginación aumentan el tiempo real sin afectar el tiempo de usuario.

En nuestros resultados, la diferencia entre tiempo real y tiempo user+sys es más notable en tamaños muy pequeños ($n=100$) y muy grandes ($n=100.000.000$), lo que sugiere que en estos casos las operaciones del sistema y la competencia por recursos tienen mayor impacto.

6.3.3. Análisis por tipo de caso

Comparando los tres tipos de casos para tamaños grandes ($n=100.000.000$):

- **Caso favorable:** 11.950s (user)
- **Caso desfavorable:** 12.242s (user)
- **Caso promedio:** 12.238s (user)

El caso favorable es ligeramente más rápido (aproximadamente un 2.4 % menos tiempo) que los otros casos, pero la diferencia no es tan significativa como se esperaría teóricamente. Esto sugiere que, en la práctica, la distribución de los caracteres en la cadena tiene un impacto limitado en el rendimiento de nuestro algoritmo.

6.3.4. Escalabilidad

Para analizar la escalabilidad, calculamos el factor de crecimiento del tiempo (user) cuando aumentamos el tamaño por un factor de 10. Este factor se obtiene dividiendo el tiempo de ejecución para un tamaño N entre el tiempo para un tamaño $N/10$. En un algoritmo perfectamente lineal, esperaríamos un factor exactamente igual a 10.

Incremento de tamaño	Favorable	Desfavorable	Promedio
1,000 / 100	1.00	-	-
10,000 / 1,000	2.00	-	2.00
100,000 / 10,000	8.00	6.00	6.50
1,000,000 / 100,000	7.63	9.92	9.62
10,000,000 / 1,000,000	9.97	10.06	9.62
100,000,000 / 10,000,000	9.83	10.23	10.17

Notas: Algunos factores no se pueden calcular exactamente debido a valores de 0.000s en tamaños pequeños.

Para tamaños pequeños, el factor de crecimiento es menor que 10, lo que indica que hay factores constantes o efectos de caché que dominan en estos tamaños. Para tamaños grandes ($n \geq 1.000.000$), el factor se aproxima a 10, lo que sugiere un comportamiento casi lineal.

7 Contraste del estudio teórico y experimental

En esta sección se comparan los resultados obtenidos en el análisis teórico de la complejidad del algoritmo con los resultados experimentales obtenidos al ejecutar el programa con diferentes tamaños de entrada.

7.1 Comparación de complejidades

En el análisis teórico se determinó que:

- El caso mejor tiene una complejidad de $O(n)$
- El caso peor tiene una complejidad de $O(n \cdot m \cdot \log n)$
- El caso promedio tiene una complejidad de $O(n \cdot m \cdot \log n)$

Si analizamos los datos experimentales, podemos calcular el factor de crecimiento entre tamaños consecutivos de entrada. Para un algoritmo con complejidad $O(n \cdot \log n)$, al multiplicar n por 10, esperaríamos que el tiempo se multiplicara aproximadamente por un factor de 10 a 13 (ya que $\log(10n) = \log(10) + \log(n) \approx 1 + \log(n)$).

Observando nuestros resultados experimentales (usando el tiempo de CPU, es decir, User), vemos que para tamaños grandes ($n \geq 1.000.000$), el factor de crecimiento se aproxima a 10, lo que indica un comportamiento más cercano a $O(n)$ que a $O(n \cdot m \cdot \log n)$.

7.2 Discrepancias observadas

Al comparar el estudio teórico con los resultados experimentales, se observan varias discrepancias:

1. **Tamaños pequeños:** Para entradas pequeñas ($n = 100$ a 10.000), el factor de crecimiento es significativamente menor que el esperado teóricamente (entre 1 y 8). Esto sugiere que otros factores, como la sobrecarga constante de las llamadas recursivas y los efectos de caché, tienen más peso que la complejidad asintótica para estos tamaños.
2. **Tamaños medianos y grandes:** Para tamaños de $1.000.000$ a $100.000.000$, el factor de crecimiento se aproxima a 10, lo que indica un comportamiento más cercano a $O(n)$ que a $O(n \cdot m \cdot \log n)$.
3. **Similitud entre casos:** Los tres tipos de casos (favorable, desfavorable y promedio) muestran tiempos muy similares, con diferencias de solo un 2-3 %. Esto contradice la expectativa teórica de que el caso favorable debería ser significativamente más rápido.

7.3 Justificación de las discrepancias

Estas discrepancias pueden explicarse por los siguientes factores:

1. **Constantes ocultas:** El análisis asintótico ignora las constantes multiplicativas, que pueden ser significativas en la práctica. La función auxiliar `contarAparicionesConsecutivas` tiene una implementación eficiente con constantes bajas.

2. **Optimizaciones del compilador:** El compilador puede realizar optimizaciones que reducen el impacto del factor logarítmico, especialmente en el caso recursivo. Las optimizaciones como el desenrollado de bucles y la eliminación de recursión de cola pueden cambiar significativamente el rendimiento.
3. **Efectos de la caché:** Para tamaños pequeños y medianos, los datos pueden caber completamente en la caché del procesador, lo que reduce significativamente los tiempos de acceso a memoria y puede enmascarar el factor logarítmico. Esto explicaría por qué el crecimiento es mucho más lento para tamaños pequeños.
4. **Localidad de referencia:** El algoritmo divide y vencerás tiene buena localidad de referencia, lo que significa que accede a datos cercanos en memoria, aprovechando la jerarquía de caché. Esto puede reducir significativamente el impacto de las operaciones de memoria, que suelen ser el cuello de botella en algoritmos que procesan grandes volúmenes de datos.
5. **Valor fijo de m:** En nuestros experimentos, el valor de m (longitud de la subcadena) se mantiene constante en 100. Dado que la complejidad teórica es $O(n \cdot m \cdot \log n)$, al mantener m constante, la complejidad efectiva es $O(n \cdot \log n)$, que para valores grandes de n se aproxima más a $O(n)$ en la práctica.
6. **Implementación específica:** La implementación concreta puede contener optimizaciones que no se consideran en el análisis teórico general. Por ejemplo, puede haber optimizaciones en el manejo de cadenas o en la detección temprana de casos que no pueden mejorar la solución actual.

7.4 Conclusiones del contraste

El estudio experimental muestra un comportamiento que difiere parcialmente del análisis teórico:

- Para tamaños pequeños ($n < 100,000$), el algoritmo crece mucho más lentamente de lo esperado, lo que se puede atribuir a efectos de caché y constantes ocultas.
- Para tamaños grandes ($n \geq 1,000,000$), el crecimiento es prácticamente lineal con un factor cercano a 10, lo que indica que el término logarítmico tiene un impacto limitado en la práctica.
- Los tres tipos de casos (favorable, desfavorable y promedio) muestran rendimientos muy similares, lo que sugiere que el algoritmo es robusto frente a diferentes distribuciones de datos.

Esto demuestra la importancia de complementar el análisis teórico con estudios experimentales para obtener una comprensión completa del rendimiento del algoritmo en situaciones reales.

La divergencia entre teoría y práctica no invalida el análisis teórico, sino que muestra cómo factores prácticos como la arquitectura del computador, la jerarquía de memoria y las optimizaciones del compilador pueden influir significativamente en el rendimiento real del algoritmo.

8 Conclusiones

8.1 Conclusiones personales

La implementación de este algoritmo de divide y vencerás ha sido una experiencia muy interesante que me ha permitido profundizar en varios aspectos importantes de la programación y el análisis de algoritmos.

Uno de los puntos más fascinantes fue descubrir la discrepancia entre el análisis teórico y el comportamiento práctico del algoritmo. Aunque teóricamente teníamos una complejidad de $O(n \cdot m \cdot \log n)$, los resultados experimentales mostraron un comportamiento prácticamente lineal para tamaños grandes de entrada. Esto me hizo reflexionar sobre la importancia de complementar el análisis teórico con estudios empíricos, ya que factores como las optimizaciones del compilador y la arquitectura del procesador pueden influir significativamente en el rendimiento real.

También fue interesante observar cómo los tres tipos de casos (favorable, desfavorable y promedio) mostraron tiempos de ejecución muy similares, con diferencias de apenas un 2-3%. Esto sugiere que el algoritmo es robusto y eficiente independientemente de la distribución de los datos de entrada, lo cual es una característica muy deseable en la práctica.

La parte más desafiante fue el diseño y ejecución de los experimentos para medir el tiempo de ejecución. Tuvimos que considerar varios factores como el tamaño de la entrada, la distribución de los datos y la precisión de las mediciones. Sin embargo, al final, ver cómo los datos experimentales confirmaban (o en algunos casos, contradecían) nuestras expectativas teóricas fue muy gratificante.

En general, este trabajo me ha ayudado a comprender mejor la importancia de combinar el análisis teórico con la experimentación práctica. A veces, la realidad puede sorprendernos y mostrarnos aspectos que el análisis teórico por sí solo no puede capturar. Esta experiencia me ha enseñado a ser más crítico con las predicciones teóricas y a valorar la importancia de la validación experimental.